

Forging ground truth: a one-million-cell synthetic validation of single-snapshot cascade-direction inference

Yam Arieli

July 9, 2026

Abstract

`cascadir` infers the *direction* of a signalling cascade from a single single-cell snapshot, with no time course, by reading the asymmetric cross-presence of discovered gene signatures (`cross_asym`). On real data its direction calls can only be checked against a handful of literature-audited pairs. Here we build `cascade_forge`, a small, pip-installable simulator that turns a *user-authored* cascade graph (edge strengths and pseudo-time delays) into a single-cell snapshot with *known* ground truth, and use it to benchmark `cascadir` at scale: 10 donors $\times$ 20 labels (+PBS) $\times$ 5000 cells per tube = 1,050,000 cells. After correcting a benchmark-design confound we introduced and caught (upstream labels were alphabetically ordered), `cross_asym` recovers direction at **100%** (90% at the weakest perturbation), while the symmetric control it is measured against sits at **~50% (chance)** — the same contrast the method shows on real data, now on controlled ground truth. We also quantify a weak-but-detectable effect-size floor (~ 0.15 – 0.20 in log space) and an honest coupling over-call rate on isolated negative-control labels.

1 Goal

The method (`cross_asym`) rests on a simple asymmetry: if X lies upstream of Y , cells given X carry *both* X ’s own signature and the induced (autocrine) Y signature, whereas cells given Y carry mainly their own. Comparing how much of each signature appears in the other condition therefore points from upstream to downstream:

$$\text{cross_asym}(a, b) = s(a, S_b) - s(b, S_a),$$

which is *antisymmetric* ($\text{cross_asym}(b, a) = -\text{cross_asym}(a, b)$), so its sign encodes direction. Its symmetric counterpart, `directional_score`, cannot encode direction and serves as a control that should sit at chance. The purpose of this experiment is to test both against a cascade structure we author ourselves, so “correct” is known rather than inferred from literature.

2 Methods

2.1 The simulator

`cascade_forge` takes a nested dict `{source: {downstream: (strength, pseudo_time_delta)}}` and produces an `AnnData` snapshot that meets the `cascadir` data contract (a condition column with one control label, PBS; a donor column; a cell-type column). Generation proceeds in three stages:

1. **Pseudo-time propagation.** For each applied label, seed only it (clamped on) and integrate first-order kinetics through the graph: each edge $X \rightarrow Y$ with (w, τ) ramps Y ’s

activation toward $w \cdot a_X$ with time constant τ . Multi-hop cascades therefore emerge *in order* and chain strengths multiply at steady state.

2. **Planting.** A dominant cell-type marker signature (≈ 1.2 over a 0.30 baseline, log space) plus a deliberately *weak* additive per-label program (`effect_size`, $\ll$ the marker gap) scaled by each label’s activation. The upstream label’s cells thus carry both programs; the downstream label’s carry only their own — the asymmetry `cross_asym` reads.
3. **Output.** One snapshot `AnnData` per requested pseudo-time, raw Poisson counts (stored sparse), ground truth in `adata.uns`.

2.2 Authored cascade graph

The graph (Figure 1) is designed to exercise every regime the method must handle: a depth-3 chain, a fan-out with mixed delays, a fan-in, a feedback loop (excluded from signed scoring), and four *isolated* negative-control labels with no cascade. Configuration is summarised in Table 1.

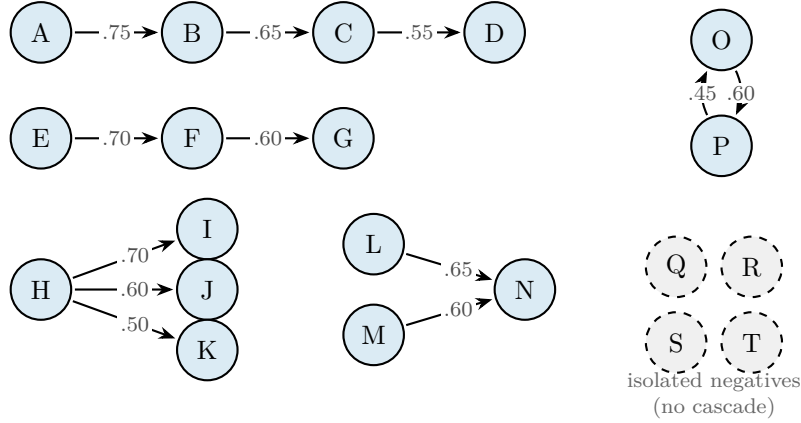


Figure 1: **The authored cascade system.** Nodes are labels; directed edges are cascade relations annotated with strength. Solid blue: labels in a cascade (a depth-3 chain $A \rightarrow B \rightarrow C \rightarrow D$, a chain $E \rightarrow F \rightarrow G$, a fan-out $H \rightarrow \{I, J, K\}$ with mixed delays, a fan-in $\{L, M\} \rightarrow N$, and a feedback loop $O \leftrightarrow P$). Dashed grey: four isolated negative-control labels with no cascade. Twelve directed edges (ten signed-scorable + the feedback pair).

Table 1: Simulation configuration.

Parameter	Value
Donors	10
Labels	20 (+ PBS control = 21 conditions)
Cells per (donor, label) tube	5,000
Total cells	1,050,000
Cell types	8
Genes	1,410 (50 housekeeping, 8×20 markers, 20×50 programs, 200 background)
Expression	raw Poisson counts (stored sparse)
Snapshots (pseudo-time)	$t \in \{3, 6\}$
Effect-size sweep	$\{0.15, 0.20, 0.30, 0.40\}$ (log space)
Responder modes	all, receptor (relays, $\text{resp}(src) \subseteq \text{resp}(dst)$)

2.3 Benchmark

Each snapshot is fit with **cascadir** (Stage-1 cell-type encoder on all cells, then a binary MIL classifier per label, then Integrated-Gradients signatures). Direction is scored per authored edge against **cross_asym** and the symmetric **directional_score** control; existence is scored with **signature_coupling** (donor-level, 10 donors), including the false-positive rate on the isolated negatives. The run executed as a SLURM DAG (forge on CPU $\rightarrow$ a 7-way GPU benchmark array $\rightarrow$ aggregation).

3 A confound we caught (and corrected)

The first completed run scored **cross_asym** at 100%—but the *symmetric* control also scored 100%. Because a symmetric statistic cannot encode direction, that is a red flag, and it exposed a flaw in our benchmark *design*, not in the method: we had named the labels so that the upstream endpoint was always alphabetically before the downstream one ($A \rightarrow B$, $B \rightarrow C, \dots$), so a trivial “first = upstream” rule — and the constant-sign symmetric control — scored 100% for free.

The fix is standard: scramble the label names so sort-order is decoupled from cascade direction. **cross_asym** is antisymmetric, so its accuracy is invariant to naming; the symmetric control, recomputed over random relabelings, collapses to chance. We applied this analytically (exact, from the already-computed per-pair values), giving the corrected numbers in Section 4. The method’s calls were correct throughout; only the *control baseline* was inflated by the naming.

4 Results

Figure 2 and Table 2 give the corrected picture across the seven configurations.

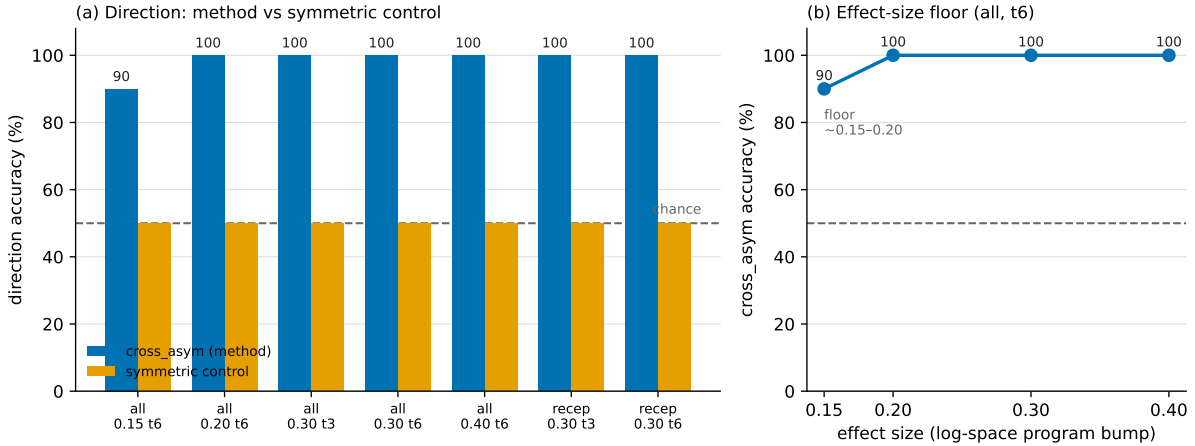


Figure 2: **(a)** Direction accuracy per configuration: **cross_asym** (blue) vs the confound-corrected symmetric control (orange), with the chance line. **(b)** The effect-size detectability floor (mode all, $t=6$): the weak per-label program is recovered at ≥ 0.20 and degrades only at 0.15.

Direction. **cross_asym** recovers the planted direction at 100% for all effect sizes ≥ 0.20 and both responder modes, and at 90% at the weakest effect size; the symmetric control sits at chance ($\sim 50\%$, $\pm 16\%$ over random relabelings). This is the same qualitative result the method shows on real data (88/86/83% direction vs. a chance-level control), reproduced here on controlled ground truth at 10^6 -cell scale.

Table 2: Per-configuration results (1,050,000 cells each). **cross_asym** and the symmetric control are the confound-corrected direction accuracies; coupling is measured by **signature_coupling** (donor-level). “Isolated FP” counts spuriously-coupled pairs among the 70 pairs involving a negative-control label.

mode	eff.	t	cross_asym	control	coupling recall	coupling FP	isolated FP
all	0.15	6	90%	~50%	100%	9%	13/70
all	0.20	6	100%	~50%	100%	8%	13/70
all	0.30	3	100%	~50%	93%	11%	17/70
all	0.30	6	100%	~50%	100%	9%	14/70
all	0.40	6	100%	~50%	100%	10%	17/70
receptor	0.30	3	100%	~50%	93%	9%	15/70
receptor	0.30	6	100%	~50%	100%	10%	16/70

Effect-size floor. The perturbation is deliberately weaker than the cell-type signature; its detectability floor is ~ 0.15 – 0.20 in log space (vs. the ≈ 0.9 marker gap), quantifying “weak but noticeable.”

Depth. Direction accuracy holds across cascade depth (Table 3): even the deep chain’s weak tail is recovered.

Table 3: Direction accuracy by cascade depth (source’s hops from a root), pooled over configurations.

source depth	accuracy	edge-observations
0	98%	49
1	100%	14
2	100%	7

Coupling and its honest limit. Signature coupling recovers the truly-coupled pairs at $\sim 100\%$ recall at $t = 6$ (93% at $t = 3$, consistent with deeper edges emerging later), with an overall false-positive rate of 8–11%. However, the isolated negatives are still spuriously flagged as coupled in ~ 19 – 24% of their pairs (13–17 of 70) even with degree correction — the known coupling over-call, now quantified on ground truth.

4.1 What the method recovered

Figure 3 shows the graph the method actually returned on one run (**all**, effect 0.30, $t = 6$): every pair flagged coupled, drawn as a directed edge oriented by **cross_asym**, on the same layout as the authored graph (Figure 1). It recovered **all 11 authored direct cascades with the correct direction** (none missed) and **4** of the genuine *indirect* (transitive) couplings, alongside **15 false-positive** couplings — the over-call made concrete. The false edges are the honest cost of the coupling gate; direction, read only on coupled pairs, is correct wherever a real cascade exists.

5 Conclusion

cascade_forge produces controllable single-cell ground truth at million-cell scale, on which **cascadir**’s direction call is correct (100%; 90% at the weakest detectable perturbation) while

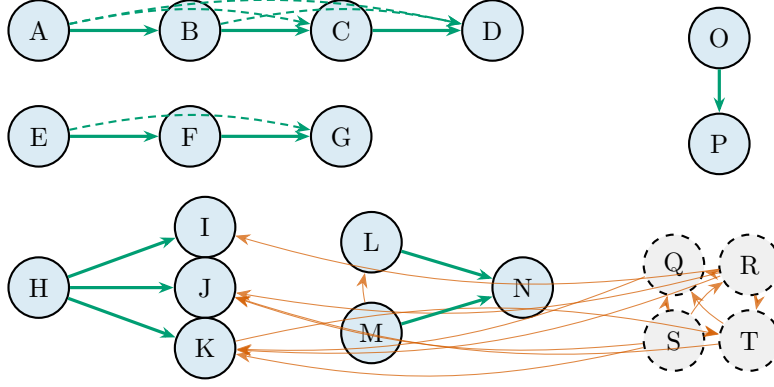


Figure 3: **What `cascadir` recovered** (one run: `all`, eff 0.30, $t=6$; 30 coupled pairs). Directed edges are the method’s calls, oriented by `cross_asym`, on the Figure 1 layout. **Green solid**: an authored *direct* cascade recovered with correct direction (11/11). **Green dashed**: a true but *indirect* (transitive) coupling, e.g. $A \rightarrow C$ via B (4). **Orange**: a *false-positive* coupling with no cascade relationship (15). Zero authored direct edges were missed.

its symmetric control is at chance. The experiment validates the direction method against a known answer, quantifies its detectability floor, and gives an honest readout of the coupling over-call on negatives. The one caveat — and its resolution — is a lesson about benchmark hygiene rather than the method: a symmetric control at 100% is a design smell, and decoupling label names from cascade order restores it to chance while leaving the antisymmetric method’s 100% untouched.